Roteiro de Desenvolvimento do Projeto – Guia para Estudantes

Algoritmos e Estruturas de Dados (2026/2027)

Este roteiro apresenta a sequência de implementação recomendada para o desenvolvimento das funções dos TADs `Primitive` e `VectorScene`.

⚠️ Princípios Obrigatoriamente a Respeitar

> 1. **Desenvolvimento e Teste Incremental por Fases:**
>   - **Não tente implementar todo o projeto de uma só vez.**
>   - Deve **completar e testar rigorosamente cada fase** (usando os testes unitários fornecidos e a ferramenta **Valgrind**) antes de avançar para a fase seguinte. A deteção precoce de falhas de lógica ou de fugas de memória poupa tempo e evita erros em cascata.
> 2. **Uso Obrigatório das Funções Auxiliares Fornecidas (Sem Código Repetido):**
>   - **Não volte a escrever funções matemáticas ou geométricas que já estão desenvolvidas.**
>   - O ficheiro `primitive.c` já inclui funções auxiliares totalmente implementadas (ex.: `math_circle_contains_point`, `math_rect_contains_point`, `math_segment_contains_point`, `math_polygon_contains_point`).
>   - As funções do TAD que terá de desenvolver (como `primitive_contains_point`) **devem obrigatoriamente invocar e reutilizar** estas funções auxiliares. A duplicação desnecessária de código será penalizada na avaliação da qualidade e clareza da solução.

FASE 1: Construtores Básicos e Gestão de Memória no TAD `Primitive`

> **Objetivo:** Compreender a estrutura de dados `Primitive` (*Tagged Union*) e a alocação e libertação básica de memória dinâmica.
> **Instrução de Teste:** Testar cada construtor individualmente e validar com **Valgrind** que a destruição (`primitive_destroy`) liberta toda a memória alocada sem deixar fugas de memória.

- [ ] **`primitive_create_rectangle`** (`primitive.c`)
- [ ] **`primitive_create_segment`** (`primitive.c`)
- [ ] **`primitive_create_polygon`** (`primitive.c`)
- [ ] **`primitive_destroy`** (`primitive.c`)

FASE 2: Manipulação do Array Dinâmico associado a um Polígono

> **Objetivo:** Compreender e utilizar o redimensionamento dinâmico de memória (`realloc`) e a reorganização de arrays em C.
> **Instrução de Teste:** Testar a adição consecutiva de múltiplos vértices para forçar a expansão de capacidade do array e verificar a remoção correta com deslocamento (*shift*).

- [ ] **`polygon_add_vertex`** (`primitive.c`)
- [ ] **`polygon_remove_vertex`** (`primitive.c`)

FASE 3: Geometria, *Axis-Aligned Bounding-Box* (AABB) e Transformações no TAD **`Primitive`**

> **Objetivo:** Implementar o recalculo preguiçoso (*lazy evaluation*) do retângulo delimitador (AABB) e as transformações geométricas.
> **Instrução de Reutilização:** A função `PrimitiveContainsPoint` **deve obrigatoriamente reutilizar** as funções auxiliares `Math...ContainsPoint` fornecidas no início de `primitive.c`.

- [ ] **`primitive_update_aabb`** (`primitive.c`)
- [ ] **`primitive_translate`** (`primitive.c`)
- [ ] **`primitive_scale`** (`primitive.c`)
- [ ] **`primitive_contains_point`** (`primitive.c`) — *Usar obrigatoriamente as funções auxiliares `Math...`!*

FASE 4: Coleção de Primitivas e Ciclo de Vida no TAD **`VectorScene`**

> **Objetivo:** Gerir a coleção de primitivas referenciadas por ponteiro (`Primitive**`).
> **Instrução de Teste:** Adicionar e remover primitivas da cena, confirmando que a eliminação da cena (`SceneDestroy`) liberta todas as primitivas em cascata sem fugas de memória.

- [ ] **`vector_scene_create`** (`vector_scene.c`)
- [ ] **`vector_scene_destroy`** (`vector_scene.c`)
- [ ] **`vector_scene_add_primitive`** (`vector_scene.c`)
- [ ] **`vector_scene_remove_primitive`** (`vector_scene.c`)
- [ ] **`vector_scene_translate_primitive`** (`vector_scene.c`)
- [ ] **`vector_scene_set_z_index`** (`vector_scene.c`)

FASE 5: Algoritmos e Otimizações

> **Objetivo:** Implementar os algoritmos sujeitos à análise de complexidade e instrumentação de métricas (`SortStats` e `PickStats`).
> **Instrução de Teste:** Executar benchmarks com cenas de teste (ex.: `exemplo_cena.vsc`) e verificar se os contadores de comparações, trocas e testes AABB são devidamente incrementados.

- [ ] **`vector_scene_sort_by_z`** (`vector_scene.c`)
- [ ] **`insertion_sort_by_z`** (`vector_scene.c`)
- [ ] **`merge_by_z`** (`vector_scene.c`)
- [ ] **`merge_sort_by_z`** (`vector_scene.c`)
- [ ] **`vector_scene_pick_top_primitive`** (`vector_scene.c`)
- [ ] **`vector_scene_render`** (`vector_scene.c`)

Quadro Resumo

Fase	Módulo	Foco Principal	Validação Recomendada
1	`Primitive`	Construção, inicialização e destruição de primitivas geométricas sem *memory leaks*.	Testes de alocação e libertação de memória + Valgrind
2	`Primitive` (Polígono)	Inserção/remoção de vértices com `realloc` e deslocamento (*shift*).	Teste de expansão de capacidade
3	`Primitive` (Geometria)	Recalculo de AABB condicional, mudança de escala, translação e pertença de ponto.	Reutilização das funções `math...`
4	`VectorScene` (Coleção)	Criação de uma cena, gestão do array de ponteiros para primitivas geométricas.	Teste de destruição em cascata
5	`VectorScene` (Algoritmos)	Algoritmos de ordenação por profundidade das primitivas da cena, *picking* de um ponto usando AABB e 2 fases, e rasterização com *Alpha Blending*.	Verificação das métricas `SortStats` / `PickStats`

Utilização de IA

A primeira versão deste documento foi desenvolvida com o auxílio do **Google Gemini.**

A versão final foi editada e validada por Joaquim Madeira.